

x86-64 Assembly Reference — Functions, ABI & Floating Point

System V AMD64 calling convention (Linux) · Intel syntax · compiled with gcc/nasm

1 Registers

General purpose (64-bit)

Reg	Role	Saved by
rax	return value, syscall #	caller
rdi	arg 1	caller
rsi	arg 2	caller
rdx	arg 3	caller
rcx	arg 4	caller
r8	arg 5	caller
r9	arg 6	caller
r10, r11	scratch	caller
rbx	general	callee
rbp	frame pointer	callee
rsp	stack pointer	callee
r12–r15	general	callee

Floating point (SSE)

Reg	Role	Saved by
xmm0	FP arg 1 / FP return	caller
xmm1–xmm7	FP args 2–8	caller
xmm8–xmm15	scratch	caller

128-bit registers; low 32 bits hold a **float**, low 64 bits hold a **double**. All xmm regs are caller-saved — nothing is preserved across a call.

al must hold the number of vector (xmm) registers used, for variadic calls like `printf`.

2 Function calls: registers, stack & the frame

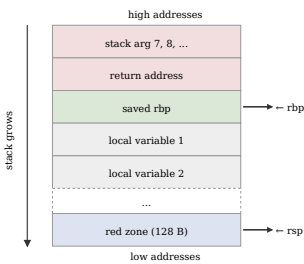
Argument passing (System V AMD64 ABI)

- First 6 integer/pointer args → `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`
- First 8 floating-point args → `xmm0`–`xmm7`
- Extra args pushed on the stack, right-to-left
- Integer/pointer return → `rax` · FP return → `xmm0`
- Stack must be 16-byte aligned at the call instruction
- 128-byte "red zone" below `rsp` usable as scratch without adjusting `rsp` (leaf functions only)

Standard prologue / epilogue

```
func:
    push    rbp
    mov     rbp, rsp
    sub     rsp, N           ; locals (N mult. of 16)
    ...
    leave           ; mov rsp,rbp ; pop rbp
    ret
```

Stack frame diagram



3 Caller-saved vs. callee-saved — a worked example

The rule

- **Caller-saved** (rax, rcx, rdx, rsi, rdi, r8–r11): a called function is free to clobber these. If the *caller* needs their values after the call, the caller must save them — typically by pushing to the stack — before the call.
- **Callee-saved** (rbx, rbp, r12–r15): if a function wants to use these, *it* must save the caller's values on entry and restore them before returning. The caller can assume they're unchanged across any call.

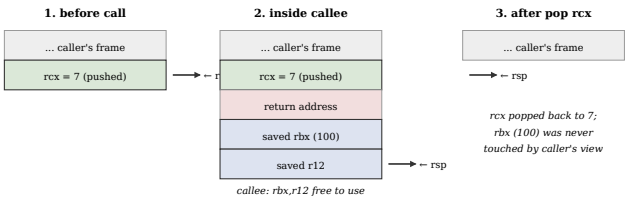
Scenario

A function caller is using *rbx* (its own data) and *rcx* (a live loop counter) when it calls callee, which internally needs *rbx* and *r12* for its own work.

```
caller:
    ...                ; rbx = 100, rcx = 7 (both live)
    push rcx           ; save caller-saved reg we still need
    call callee
    pop rcx            ; restore it – rcx is back to 7
    ...                ; rbx is untouched – callee guaranteed it

callee:
    push rbx           ; save callee-saved reg before using it
    push r12
    mov  rbx, 42        ; free to use rbx now
    mov  r12, 5
    ...
    pop  r12            ; restore in reverse order
    pop  rbx
    ret
```

Stack at each stage



Register values through the call			
stage	rbx	rcx	r12
before call	100	7	—
inside callee, after its pushes	100 (saved)	7 (saved)	saved (old val)
inside callee, mid-work	42	7 (untouched)	5
after callee's pops, before ret	100 (restored)	7	restored
back in caller, after pop rcx	100	7	n/a to caller

Key takeaway: *rbx* is never explicitly saved by the caller — it doesn't need to be, because the callee contract guarantees it. *rcx* is explicitly saved by the caller, because nothing obliges the callee to preserve it.

4 Floating point (SSE) instructions

Move

Instr	Meaning
movss dst, src	move scalar single (32-bit float)
movsd dst, src	move scalar double (64-bit)
movaps / movapd	move aligned packed (16B aligned!)
movups / movupd	move unaligned packed

Arithmetic — pattern: op dst, src → dst = dst op src

Instr	Meaning
addss / addsd	add (single / double)
subss / subsd	subtract
mulss / mulsd	multiply
divss / divsd	divide
sqrts / sqrtsd	square root

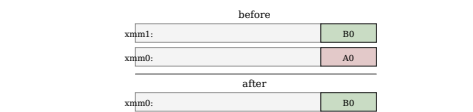
Suffix key: **ss**=scalar single, **sd**=scalar double, **ps**=packed single, **pd**=packed double.

XMM register layout (128 bits, 4 lanes of 32 bits)



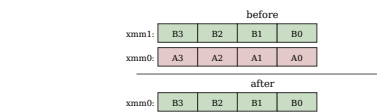
Each lane is 32 bits — one float. Two adjacent lanes (0+1, or 2+3) together form one 64-bit double. Lane 0 is the **scalar** lane: it is the only one touched by ss/sd instructions.

movss xmm0, xmm1



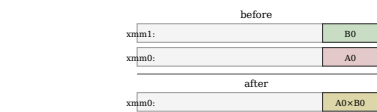
Only lane 0 (32 bits) moves. Lanes 1–3 of xmm0 are **zeroed** (reg-to-reg form).

movaps xmm0, xmm1



All 128 bits (4 lanes) copied at once. Both operands must be 16-byte aligned in memory form, or it faults.

mulss xmm0, xmm1



Lane 0 only: $xmm0 \leftarrow xmm0 \times xmm1$. Upper lanes of xmm0 are left **unchanged** (unlike movss).

Packed forms (mulps, addpd, ...) apply the same op to every lane independently and in parallel — e.g. mulps xmm0, xmm1 does all four lane multiplies (A0×B0, A1×B1, A2×B2, A3×B3) in one instruction.

Worked example — double scale(double a, int b) { return b * a; }

```
cvtsi2sd  xmm1, edi    ; convert int arg 'b' (in edi) to double
mulsd     xmm0, xmm1    ; xmm0 (double 'a') *= xmm1
ret
```

Source: CS4400 (Utah), SSE floating point slides — adapted to Intel syntax.

5 Linux x86-64 syscalls

Convention: syscall number in *rax*; args in *rdi*, *rsi*, *rdx*, *r10*, *r8*, *r9* (note: **r10**, not *rcx* — *rcx* is clobbered by the `syscall` instruction itself); invoke with `syscall`; return value in *rax* (negative = `-errno`).

rax	name	rdi	rsi	rdx
0	read	fd	buf*	count
1	write	fd	buf*	count
2	open	path*	flags	mode
3	close	fd	—	—
9	mmap	addr*	len	prot
12	brk	addr*	—	—
60	exit	status	—	—
231	exit_group	status	—	—

Example — write("hi\n") then exit(0)

```
section .data
    msg db "hi", 10

section .text
    global _start
_start:
    mov rax, 1          ; syscall: write
    mov rdi, 1          ; fd = stdout
    lea rsi, [msg]      ; buf
    mov rdx, 3          ; count
    syscall

    mov rax, 60         ; syscall: exit
    xor rdi, rdi        ; status = 0
    syscall
```

Full syscall list with all argument registers: syscalls.w3challs.com/?arch=x86_64